

Constructors

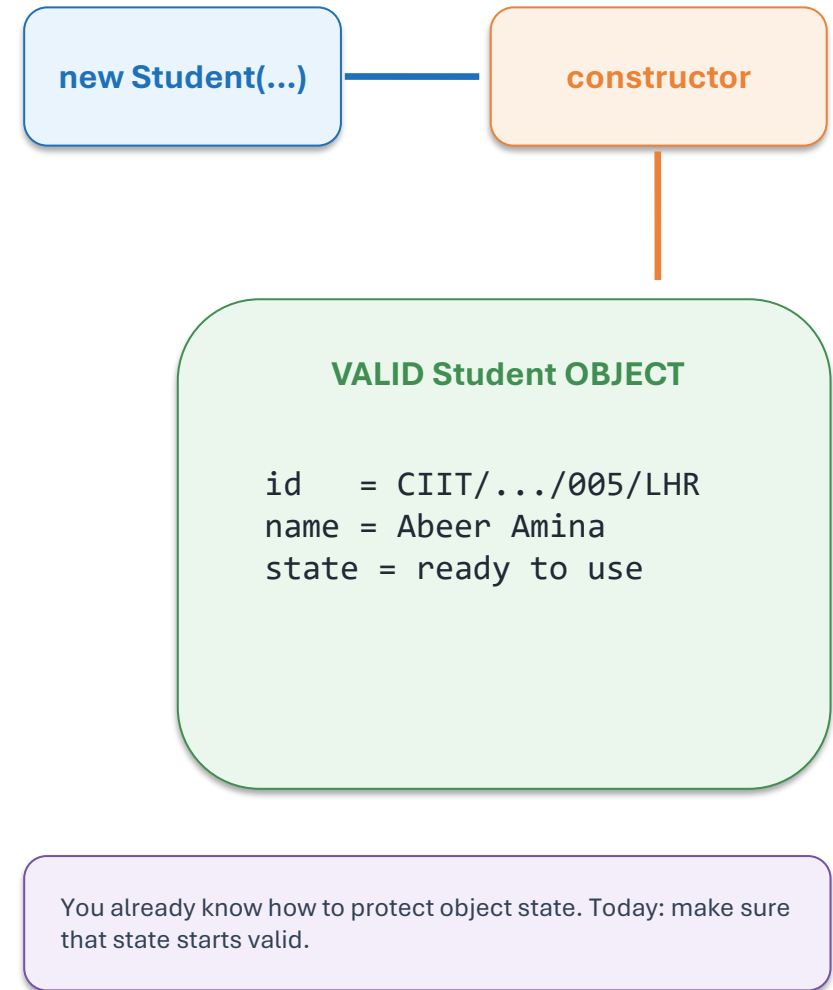
From encapsulation to valid object creation

CSC241 - Object-Oriented Programming

Fall 2026 | COMSATS University Islamabad, Lahore Campus

Dr. Muhammad Shahid Bhatti • Assistant Professor • Office H-17

Builds on: classes • objects • references • overloading • access control • encapsulation



Bridge from Quiz 1 and Lectures 3–5

Everything covered so far leads to one question: how should a valid object begin its life?

Already covered

- Classes and objects
- Primitive vs. reference variables
- Fields, methods, and this
- Method overloading

Lecture 5 → Quiz 1

- public / private / protected / default
- Private fields + controlled methods
- Encapsulation and information hiding
- Class invariants: what “valid” means

Lecture 6: constructors

- Give required values at creation time
- Choose among overloaded constructors
- Reuse initialization with this(...)
- Leave every successful object valid



Key transition: encapsulation protects an object after creation; a constructor establishes the state it starts with.

Lecture 6 scope from the CDF

Liang, Chapter 8 — Constructors

01

No-arg, parameterized, default, and copy-constructor pattern

02

Constructor overloading — connected to method overloading

03

Using `this.field` when parameters shadow fields

04

Constructor chaining with `this(...)`

05

Field initialization → selected constructor body

06

Constructor access and object-creation contracts

07

Establishing valid state and enforcing invariants

Teaching sequence: predict → code → compile → run → explain. Every new idea reuses concepts students already know.

Learning outcomes

By the end of the lecture, you should be able to...

Recognize constructor syntax
and distinguish it from an ordinary method.

Choose a constructor form
no-arg, parameterized, overloaded, or copy
pattern.

Trace this() chains
and explain why the first-statement rule exists.

Connect to prior topics
default values → initializers → constructor body.

Protect invariants
reject invalid starting states at construction
time.

Debug compiler errors
caused by missing/duplicate/malformed
constructors.

Target skill: read a new-expression and explain which constructor runs, what values flow in, and why the returned object is valid.

From Quiz 1 to constructors: the “partly initialized object” problem

Encapsulation controls how state changes; constructors control whether required state exists from the start.

SETTER-BY-SETTER INITIALIZATION

```
Equipment e = new Equipment();
```

```
// object exists, but required state is missing  
e.setId(101);
```

```
// still incomplete here  
e.setName("Arduino Kit");  
e.setPrice(12000);
```

Risk: another method may run while required fields are still missing or invalid.

CONSTRUCTOR-BASED INITIALIZATION

```
Equipment e = new Equipment(  
    101,  
    "Arduino Kit",  
    12000  
);
```

```
// reference points to an initialized object
```

Better: required values cross one controlled boundary when the object is created.

What is a constructor?

A special member used to initialize a newly created object.

CONSTRUCTOR ANATOMY

```
public class Student {  
    private String id;  
    private String name;  
  
    public Student(String id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

Same name

Constructor name must exactly match the class name.

No return type

Not even void. Adding void makes it a method.

Runs with new

It executes as part of object creation.

Can take parameters

Parameters describe what data is required to initialize the object.

A constructor is not “called like a normal method” on an already existing object.

Constructor vs. ordinary method

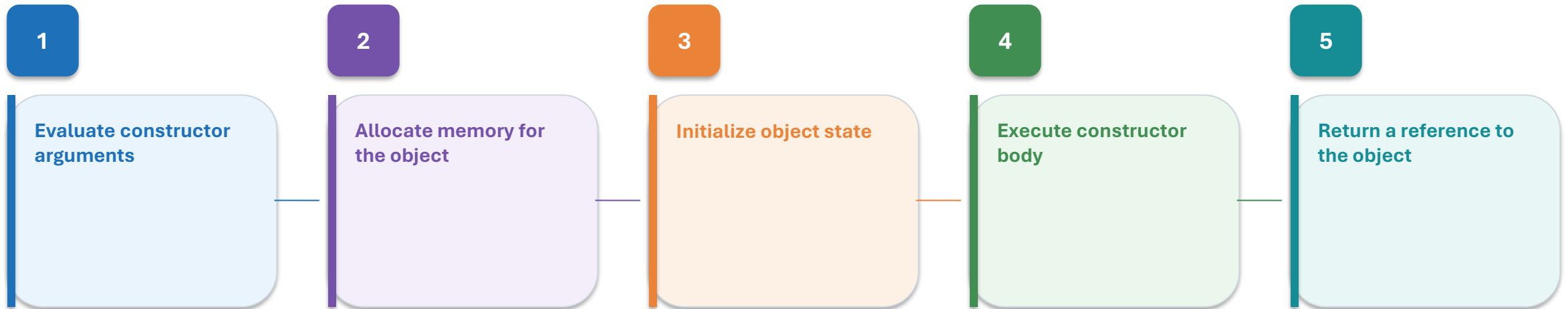
They look similar syntactically, but they serve different roles.

Aspect	Constructor	Method
Purpose	initialize a new object	perform behavior on an object
Name	must match class name	any valid method name
Return type	none	required, including void
Typical invocation	<code>new Student(...)</code>	<code>student.summary()</code>
Can overload?	yes	yes
Called repeatedly on same object?	no	yes

Quick check: `public void Student()` is a method named `Student` — it is not a constructor.

What happens when we write new Student(...)?

A useful conceptual model for tracing object creation.



```
Student s = new Student("CIIT/SP26-BAI-010/LHR", "Ali Ishtiaq");
```

Connect to Lecture 3: `s` is a reference variable. The `new`-expression creates the object, runs a matching constructor, then yields a reference to that object.

Live coding 1 — create our first constructor

Reuse Lecture 5: keep fields private. Now move required initialization from a repair sequence of setters into the constructor.

1 Predict

What happens if a constructor is missing? What values would the fields hold?

2 Code

Add Student(String id, String name).
Assign the incoming values to fields.

3 Compile

```
javac Student.java
StudentDemo.java
```

4 Run & explain

```
java StudentDemo
```

Student.java

```
public class Student {
    private String id;
    private String name;

    public Student(String id, String name) {
        this.id = id;
        this.name = name;
    }

    public String summary() {
        return id + " | " + name;
    }
}
```

StudentDemo.java

```
public class StudentDemo {
    public static void main(String[] args) {
        Student s = new Student(
            "CIIT/SP26-BAI-005/LHR",
            "Abeer Amina"
        );
        System.out.println(s.summary());
    }
}
```

Explicit no-argument constructor

A constructor with zero parameters that you write yourself.

EXPLICIT NO-ARG CONSTRUCTOR

```
public class CourseSection {  
    private String code;  
    private int capacity;  
  
    public CourseSection() {  
        code = "TBA";  
        capacity = 30;  
    }  
}  
  
CourseSection section = new CourseSection();
```

When useful

The domain has a sensible valid default state.

When suspicious

Required identity/data cannot be meaningfully invented.

Access-control connection

Because this constructor is public, client code can call `new CourseSection()`. Constructor accessibility is controlled by the same access-modifier ideas learned in Lecture 5.

“No-argument constructor” describes the parameter list. “Default constructor” has a more specific compiler-generated meaning in Java.

The compiler-provided default constructor

Java inserts one only when you declare no constructors at all.

NO CONSTRUCTOR DECLARED

```
public class Room {  
    private int seats;  
}  
  
Room r = new Room();    // compiles
```

Conceptually

The compiler supplies a no-argument constructor that invokes the superclass constructor. You did not write it.

ANY CONSTRUCTOR DECLARED

```
public class Room {  
    private int seats;  
  
    public Room(int seats) {  
        this.seats = seats;  
    }  
}  
  
new Room();    // ERROR
```

Rule to remember: once you declare any constructor, Java does not automatically add a zero-argument constructor for you.

Predict before compiling

Which object-creation statements compile?

COMPILER PREDICTION

```
class Student {  
    private String id;  
    private String name;  
  
    public Student(String id, String name) {  
        this.id = id;  
        this.name = name;  
    }  
}
```

- A) new Student();
- B) new Student("S01", "Sara");
- C) new Student("S01");
- D) new Student("S01", "Sara", 20);

A

ERROR — no zero-arg constructor

B

COMPILES — exact match

C

ERROR — one argument has no match

D

ERROR — three arguments have no match

Ask students for the answer before running javac.

Parameterized constructors: primitive and reference values can both flow in

Required state can be supplied at the moment of creation using parameters of appropriate types.

MIXED PARAMETER TYPES

```
public CourseSection(String code, int capacity) {  
    this.code = code;    // reference value  
    this.capacity = capacity; // primitive value  
}
```

```
CourseSection oop = new CourseSection(  
    "CSC241", 40  
);
```

Types still matter

String code receives a reference value;
int capacity receives a primitive value.
Java passes both argument values by value.

Creation documents intent

new CourseSection("CSC241", 40)
makes the required starting
information visible at the call site.

Constructor contract

“To create a CourseSection, provide a code and a valid capacity.” The parameter list expresses this creation contract.

A good constructor asks for what the object must know immediately — not everything it may ever know.

Why this.field matters

Constructor parameters often use the same meaningful names as the fields they initialize.

WRONG: PARAMETER ASSIGNED TO ITSELF

```
private String name;

public Student(String name) {
    name = name;
}
```

Without this, the nearest local name wins; the field remains unchanged.

RIGHT: FIELD ← PARAMETER

```
private String name;

public Student(String name) {
    this.name = name;
}
```

this refers to the current object being initialized.

Live coding 2 — create several real class objects

Use names and IDs from the class roster; do not add artificial academic attributes.

StudentDemo.java

```
Student s1 = new Student(  
    "CIIT/SP26-BAI-005/LHR",  
    "Abeer Amina"  
);  
  
Student s2 = new Student(  
    "CIIT/SP26-BAI-010/LHR",  
    "Ali Ishtiaq"  
);  
  
Student s3 = new Student(  
    "CIIT/SP26-BAI-024/LHR",  
    "Hania"  
);
```

Prediction 1

How many distinct objects are created?

Prediction 2

How many times does the constructor execute?

Prediction 3

Do s1, s2 and s3 store objects or references?

Compile: javac Student.java StudentDemo.java

From method overloading to constructor overloading

You already know compile-time overload selection. Constructors use the same idea, but each overload creates a valid starting state.

KNOWN IDEA → NEW USE

```
// Lecture 4: method overloading  
void search(int id) { ... }  
void search(String name) { ... }
```

```
// Lecture 6: constructor overloading  
Equipment() { ... }  
Equipment(int id) { ... }  
Equipment(int id, String name) { ... }
```

Same class name

Every constructor has the class name: Equipment.

Different parameter lists

As with overloaded methods, parameter count, types, and order let the compiler choose the matching constructor.

One design responsibility

Every constructor overload must still leave a valid Equipment object.

Transfer your Lecture 4 reasoning directly: overload selection happens at compile time.

How the compiler resolves overloaded constructors

Parameter count, types, and order define the constructor signature.

CourseSection()

`new CourseSection()`

selected by argument list

CourseSection(String)

`new CourseSection("CSC241")`

selected by argument list

CourseSection(String, int)

`new CourseSection("CSC241", 40)`

selected by argument list

CourseSection(int, String)

`new CourseSection(40, "CSC241")`

selected by argument list

Same overload-resolution rules as methods: exact matches are preferred; permitted primitive widening can be used when needed. Example: with `Device(long)` and `Device(double)`, `new Device(10)` selects `Device(long)`.

What does NOT create a distinct constructor overload?

The compiler needs a different parameter list — changing labels or meaning is not enough.

DUPLICATE SIGNATURE

```
public Student(String id, String name) { ... }  
  
// ERROR: same parameter types in same order  
public Student(String name, String id) { ... }  
  
// parameter names do not affect signature
```

String, String vs. String, String → duplicate, even if parameter names are swapped.

RETURN TYPE CHANGES THE MEMBER KIND

```
public Student(String id, String name) { ... }  
  
// also NOT a constructor  
public void Student(String id, String name) { ... }  
  
// this is an ordinary method
```

void Student(...) can coexist because it is a method — but it usually signals a mistake.

Design constructors around meaningful starting states

More overloads are not automatically better.

Required data

Ask for values without which the object cannot be valid.

Sensible defaults

Default only values for which the domain has a clear, safe starting choice.

Avoid ambiguous APIs

Multiple same-typed parameters can make calls hard to read and easy to reverse.

CLEAR

```
new CourseSection("CSC241", 40);
```

WHICH STRING IS WHICH?

```
new Student("Ali Ishtiaq", "CIIT/SP26-BAI-010/LHR");
```

At this stage, keep signatures simple and domain-focused. Later, richer patterns can improve readability further.

Constructor chaining with this()

Delegate from one constructor to another constructor in the same class.

CHAIN TOWARD ONE “FULL” CONSTRUCTOR

```
public CourseSection() {  
    this("TBA", 30);  
}  
  
public CourseSection(String code) {  
    this(code, 30);  
}  
  
public CourseSection(String code, int capacity) {  
    this.code = code;  
    this.capacity = capacity;  
    this.enrolled = 0;  
}
```

Why chain?

Avoid repeating default assignments and validation logic in every constructor.

Where does this(...) go?

Inside a constructor. It targets another constructor of the same class.

Design payoff

One central constructor becomes the single source of initialization truth.

Rules for this(...) constructor calls

Java makes constructor delegation intentionally strict.

this(...) IS NOT FIRST

```
public CourseSection(String code) {  
    System.out.println("creating");  
    this(code, 30);    // ERROR  
}
```

VALID

```
public CourseSection(String code) {  
    this(code, 30);    // first statement  
    System.out.println("created");  
}
```

Rule 1

this(...) must be the first statement.

Rule 2

A constructor can directly delegate to only one other constructor.

Rule 3

Chains must terminate; recursive constructor invocation is illegal.

Live coding 3 — refactor duplicated constructors

Write the duplicated version first, then remove repetition with this().

BEFORE: DUPLICATION

```
public CourseSection() {
    code = "TBA";
    capacity = 30;
    enrolled = 0;
}

public CourseSection(String code) {
    this.code = code;
    capacity = 30;
    enrolled = 0;
}

public CourseSection(String code, int capacity) {
    this.code = code;
    this.capacity = capacity;
    enrolled = 0;
}
```

AFTER: ONE INITIALIZATION PATH

```
public CourseSection() {
    this("TBA", 30);
}

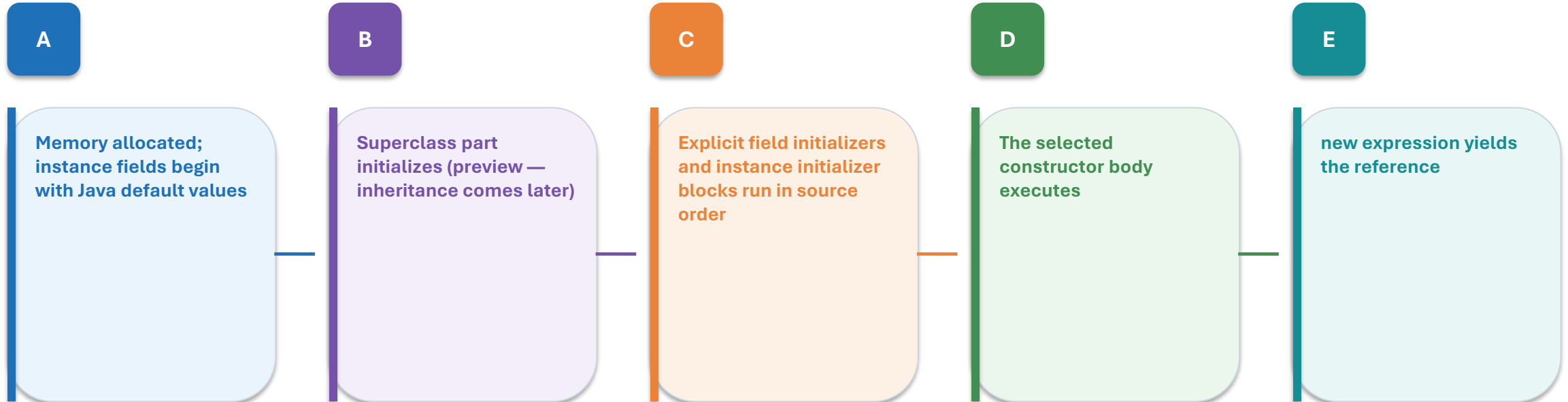
public CourseSection(String code) {
    this(code, 30);
}

public CourseSection(String code, int capacity) {
    this.code = code;
    this.capacity = capacity;
    this.enrolled = 0;
}
```

Ask: if validation is added tomorrow, which version is safer to maintain?

Object initialization order

Trace the phases rather than memorizing scattered rules.



For today, focus on the current class: default field values → explicit initializers/instance blocks → constructor body. The superclass step becomes important in Lecture 15.

Default values, field initializers, and constructor assignments

Three different moments can affect an instance field.

THREE SOURCES OF INITIAL STATE

```
class CourseSection {  
    private int enrolled;    // default 0  
    private int capacity = 30; // explicit initiali  
  
    public CourseSection(int capacity) {  
        this.capacity = capacity;  
    }  
}
```

1 Java default

Before your initialization code: int → 0, boolean → false, references → null.

2 Field initializer

Runs for each newly created object before the constructor body.

3 Constructor body

Can overwrite an earlier default/initializer with call-specific data.

new CourseSection(45) ends with capacity = 45 and enrolled = 0. The constructor assignment replaces the earlier capacity = 30 initializer.

Common mistakes at this stage

Use what you already know about access control, overloading, and references to diagnose constructor code.

FIND THE PROBLEMS

```
class Student {  
    private String name;  
  
    public void Student(String name) {  
        this.name = name;  
    }  
}
```

```
Student s = new Student();  
s.name = "Sara";
```

Mistake 1

`void Student(...)` is an ordinary method, not a constructor.

Mistake 2

If no constructor is declared, Java may provide a default no-arg constructor — so `new Student()` can still compile here.

Mistake 3

`s.name` fails outside `Student` because `name` is private. Use the class's public API instead.

Predict the initialization trace — no new syntax required

Use field initializers + the constructor body. Predict first; compile second.

InitTrace.java

```
public class InitTrace {
    private int capacity = 30;

    public InitTrace(int capacity) {
        System.out.println(
            "1) before assignment: " + this.capacity);
        this.capacity = capacity;
        System.out.println(
            "2) after assignment: " + this.capacity);
    }

    public static void main(String[] args) {
        InitTrace x = new InitTrace(40);
        System.out.println(
            "3) after new: " + x.capacity);
    }
}
```

First line?

What value came from the field initializer?

Second line?

What changes after `this.capacity = capacity`?

Final line?

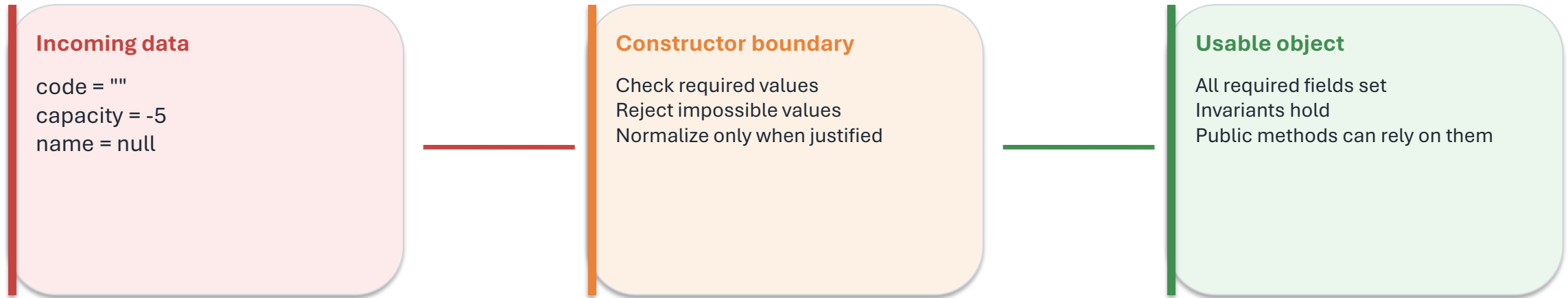
What value is stored in the object after `new` completes?

Actual output:

```
1) before assignment: 30
2) after assignment: 40
3) after new: 40
```

Constructors should establish a valid object state

Lecture 5 defined valid state. Lecture 6 makes the constructor responsible for establishing that state before client code receives the reference.



A constructor is not just a place to copy parameters into fields. It is the first opportunity to enforce the object's rules.

Validate required data inside the constructor

Reject bad starting states rather than creating an object that must be repaired later.

VALIDATING CONSTRUCTOR

```
public Student(String id, String name) {  
    if (id == null || id.isBlank()) {  
        throw new IllegalArgumentException(  
            "id is required");  
    }  
  
    if (name == null || name.isBlank()) {  
        throw new IllegalArgumentException(  
            "name is required");  
    }  
  
    this.id = id;  
    this.name = name;  
}
```

Why IllegalArgumentException?

The caller supplied an argument that violates the constructor contract.

Course connection

Formal exception handling comes later; here we use a standard runtime exception to express invalid input.

Key point

Validate before assigning when possible so the constructor's intent is easy to read.

Enforcing class invariants during construction

The constructor should establish conditions that later methods are allowed to assume.

CourseSection invariant 1

code must be nonblank

Invariant 2

$1 \leq \text{capacity} \leq 100$

Invariant 3

$0 \leq \text{enrolled} \leq \text{capacity}$

```
public CourseSection(String code, int capacity) {  
    if (code == null || code.isBlank())  
        throw new IllegalArgumentException("code required");  
    if (capacity < 1 || capacity > 100)  
        throw new IllegalArgumentException("capacity must be 1..100");  
    this.code = code;  
    this.capacity = capacity;  
    this.enrolled = 0;  
}
```

After successful construction, later methods can assume capacity is valid and enrolled starts at zero.

What if construction fails?

An exception prevents normal completion of the constructor call.

INVALID ARGUMENT

```
Student s = new Student(  
    "",  
    "Demo Student"  
);  
  
System.out.println("created");
```

For now, focus on the design meaning. We study try/catch and propagation formally in Lecture 29.

Constructor starts

Validation checks the blank ID.

Exception is thrown

The constructor does not complete normally.

No usable Student is returned

The new expression does not yield a normally constructed object to s.

Copy-constructor pattern in Java

A useful constructor pattern: create a new object from the state of another object of the same class.

COPY-CONSTRUCTOR PATTERN

```
public Student(Student other) {  
    this(other.id, other.name);  
}  
  
Student original = new Student(  
    "CIIT/SP26-BAI-005/LHR",  
    "Abeer Amina"  
);  
  
Student copy = new Student(original);
```

Why use this(other...)?

Reuse the same validation and initialization path as ordinary construction.

Identity note

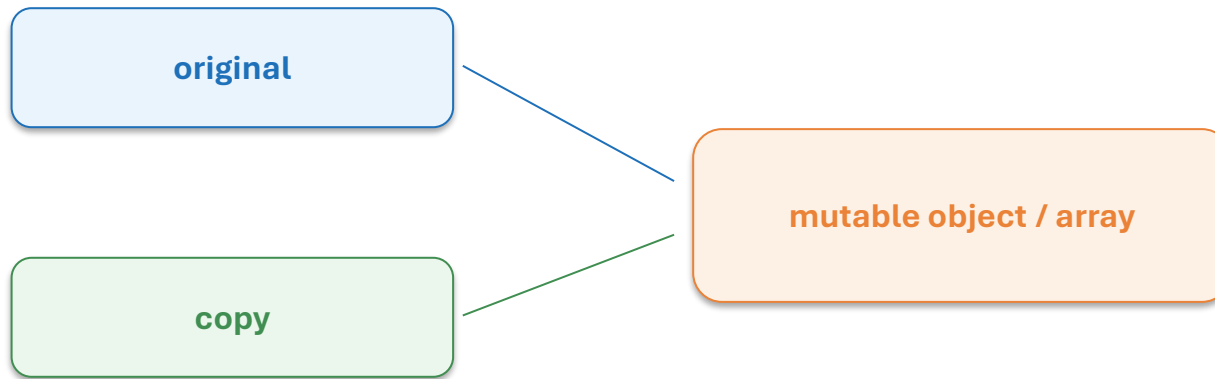
original and copy are two distinct objects even if their field values initially match.

Terminology

This is a common Java design pattern, not compiler-generated language magic.

Copying fields: values vs. references

This directly reuses your Quiz 1 idea: copying a reference value is not the same as copying the object it refers to.



Strings are immutable, so sharing a String reference does not create the same mutation risk as sharing a mutable object.

Simple assignment

Copies the reference value. Both objects may then refer to the same mutable object.

Independent copy

Requires creating/copying the referenced mutable object when the design demands independence.

Preview

Composition and Java cloning are covered later; today, recognize that copying a reference is not copying the referenced object.

Live coding 4 — add and test a copy constructor

Make the distinction between equal state and distinct object identity visible.

Student.java — ADDITIONS

```
public Student(Student other) {
    this(other.id, other.name);
}

public void rename(String name) {
    if (name == null || name.isBlank())
        throw new IllegalArgumentException();
    this.name = name;
}

public String getName() {
    return name;
}
```

PREDICT → COMPILE → RUN

```
Student original = new Student(
    "TEST-001", "Demo Student");

Student copy = new Student(original);
copy.rename("Copied Student");

System.out.println(original.getName());
System.out.println(copy.getName());
System.out.println(original == copy);
```

Expected: Demo Student | Copied Student | false

Compiler clinic + quick quiz

Decide whether each statement is valid before discussing the reason.

1

```
public void Student() { }
```

Answer

Method, not constructor

2

```
this(id, name); after another statement
```

Answer

Compile error: this(...) must be first

3

```
new Student() when only Student(String,String) exists
```

Answer

Compile error: no matching constructor

4

```
Student(String a,String b) + Student(String b,String a)
```

Answer

Duplicate signature

5

```
s.name = "Sara"; // name is private in Student
```

Answer

Compile error outside Student: private field is not directly accessible

Classroom move: ask for “compile / not compile” first; only then ask for the Java rule.

Lecture 6 recap & exit ticket

Constructors connect nearly every concept you have learned so far.

Constructor syntax

same name as class; no return type

No-arg vs. default

explicit zero-arg ≠ compiler-provided default

Overloading

same compile-time idea as method overloading

Reference + this.field

reference receives the object; this.field selects its field

this(...)

constructor delegation; must be first

Encapsulation + invariant

successful construction leaves a valid starting state

Exit ticket

1. Why does adding `Student(String id, String name)` change whether new `Student()` compiles?
2. How is constructor overloading similar to the method overloading you already learned?
3. Give one invariant that a constructor should establish before returning an object reference.

NEXT
Lecture 7
Static Members & Memory Model